

Lab 3: Architecture Justification

Sushanth S Rao, PES1UG24CS485, BPS54. Problem Statement #54: Local Sports Tournament & Bracket Manager

We chose Layered Architecture for the Local Sports Tournament & Bracket Manager System.

Architectural choice. The system is organised as a Presentation layer (Web UI), an Application layer (Registration, Bracket Engine, Score, Standings, Notification and Auth services) and a Data layer (database). Each layer talks only to the layer next to it, through the interfaces shown in the component diagram.

Reasons for the choice

1. The system's work follows a fixed pipeline: a captain registers, the bracket is generated, the director enters a score, the winner advances and the standings update. Each of these steps maps cleanly to one service, so a layered design gives every requirement (FR-001 to FR-005) a single, obvious home. This also keeps the design easy to explain, build and test as an individual project.
2. Microservices would add network calls, separate deployments and data-consistency problems that this small, single-venue system does not need, and Client-Server alone does not say how the server's logic is organised. Layering keeps the services in one deployable application while still separating them behind interfaces, so a service such as the Bracket Engine can be replaced or fixed (for example, the odd-team bye bug) without touching the UI or database.

Security advantage. All requests must pass through the Application layer, where the Auth Service checks the user's role before any action runs. Only the Tournament Director can submit scores or generate brackets, and a Team Captain can only change their own registration. The database is reachable only through the services via the IData interface, so a browser or spectator can never query or change tournament data directly. Score and registration input is validated in the service layer (for example, rejecting negative scores) before it is stored.

Performance benefit. The services run inside the same application, so a score submission triggers bracket advancement and the standings update through in-process calls rather than network hops. This keeps the delay small enough to meet NFR-001 (standings recalculated within 200 ms) and FR-004 (winner advanced within 5 seconds). Because the Standings Service owns its own queries, they can be indexed or cached without changing other layers.

Trade-off accepted. A layered application scales as one unit and has a single point of failure. For a local tournament with a few hundred users this is acceptable, and NFR-002 (99.5% uptime) is addressed with monitoring and backups instead of a distributed design.